

Physics-Informed Deep Inverse Reconstruction for Lensless Phase-Mask Imaging

A self-contained, runnable tutorial and notebook.

This notebook is a complete implementation of a lensless computational imaging pipeline. It models how a mask-based ("DiffuserCam"-style) lensless camera forms an image, and compares three ways of inverting that process to recover the scene:

1. **Wiener deconvolution** — classical, closed-form, fast.
2. **ADMM with a Total Variation (TV) prior** — classical, iterative, model-based.
3. **Unrolled ADMM network** — physics-informed deep learning, where the *structure* of ADMM is kept but the hand-designed TV proximal step is replaced by a learned CNN denoiser, and hyperparameters (like the penalty ρ) become learnable.

Everything below — PSF simulation, the forward optical model, classical solvers, the neural network, training loop, and evaluation — is implemented **inline in this notebook**. There is no external repo to clone and no `src/` package to import: just run the cells top to bottom.

What you'll find in each part :)

Part	Content
0	Setup (no repo clone needed – pure pip installs)
1	Point Spread Function (PSF) simulation for a random phase mask
2	The forward imaging model $y = \text{PSF} * x + n$ (FFT-based convolution)
3	Classical reconstruction: Wiener deconvolution and ADMM + TV
4	Physics-informed deep learning: the Unrolled ADMM network, training it from scratch
5	Loss functions & quality metrics (PSNR, SSIM, TV) explained and visualized
6	Full side-by-side comparison + summary

> Runtime: with the default image size (64×64) this notebook trains and evaluates in a few minutes on a CPU runtime, and even faster on a GPU runtime. Increase `IMG_SIZE` and training epochs for higher-fidelity results if you have more time/compute.

Part 0 — Setup

No extra ordinary requirements

```
In [ ]: # If a package is missing (e.g. running outside Colab), this installs it quietly.
import importlib, subprocess, sys

def _ensure(pkg, import_name=None):
    import_name = import_name or pkg
    try:
        importlib.import_module(import_name)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

for pkg in ["torch", "numpy", "matplotlib"]:
    _ensure(pkg)

print("Environment ready.")
```

Environment ready.

```
In [ ]: import math
import time
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
for pkg in ["torch", "numpy", "matplotlib", "scikit-image"]:
    _ensure(pkg, "skimage" if pkg == "scikit-image" else pkg)

plt.rcParams['figure.figsize'] = (12, 4)
plt.rcParams['figure.dpi'] = 110
plt.rcParams['image.cmap'] = 'gray'

SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch {torch.__version__} | device = {device}")

# ---- Global experiment settings (feel free to tune) ----
IMG_SIZE = 128 # scene / measurement resolution (increase to 96/128 for nice
NOISE_STD = 0.003 # sensor (Gaussian) noise standard deviation
PSF_FEATURE_SIZE = 4.0 # controls the "grain" of the simulated phase-mask PSF (sma
```

PyTorch 2.11.0+cu128 | device = cuda

Part 1 Simulating a Phase-Mask PSF

A lensless camera replaces the lens with a **phase mask**: a thin optical element that imparts a spatially-varying phase delay on incoming light. Because there's no lens to focus light to a point, a single point source in the scene produces a complex speckle/caustic pattern on the sensor — the camera's **Point Spread Function (PSF)**.

We simulate this physically: start from a random phase screen, upsample it smoothly (real phase masks vary smoothly at the micro-feature scale), treat it as a complex field $\exp(i\phi)$, and propagate it to the far field with a Fourier transform (a standard Fraunhofer-diffraction approximation). The **squared magnitude** of that propagated field is the sensor-plane intensity pattern — our PSF.

The forward optical model throughout this notebook is:

$$y = \text{PSF} \circledast x + n$$

i.e. the sensor measurement is the scene convolved with this PSF, plus noise.

```
In [ ]: def generate_phase_mask_psf(size=64, feature_size=4.0, seed=0):
    '''Simulate a diffuser / phase-mask PSF via Fraunhofer propagation of a random

    A coarse random phase pattern is generated and smoothly upsampled (mimicking the
    correlation length of a real diffuser's micro-structure), then propagated to the
    field via an FFT. The resulting intensity pattern is the simulated PSF.
    ...

    rng = np.random.RandomState(seed)
    low_res = max(4, int(size / feature_size))
    phase_low = rng.uniform(0, 2 * np.pi, size=(low_res, low_res)).astype(np.float32)

    phase_t = torch.from_numpy(phase_low).float()[None, None]
    phase_full = F.interpolate(phase_t, size=(size, size), mode='bicubic',
                              align_corners=False)[0, 0].numpy()

    field = np.exp(1j * phase_full)
    far_field = np.fft.fftshift(np.fft.fft2(field))
    psf = np.abs(far_field) ** 2
    psf = psf / psf.sum()
    return psf.astype(np.float32)

def gaussian_mixture_psf(size=64, num_components=40, seed=0):
    '''A softer, more 'lens-like' PSF baseline: a sum of randomly placed 2D Gaussians

    rng = np.random.RandomState(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    psf = np.zeros((size, size), dtype=np.float32)
    for _ in range(num_components):
        cy, cx = rng.uniform(size * 0.3, size * 0.7, size=2)
        sigma = rng.uniform(1.0, 3.0)
        amp = rng.uniform(0.5, 1.0)
        psf += amp * np.exp(-((yy - cy) ** 2 + (xx - cx) ** 2) / (2 * sigma ** 2))
    psf = psf / psf.sum()
    return psf.astype(np.float32)

psf_phase = generate_phase_mask_psf(IMG_SIZE, PSF_FEATURE_SIZE, seed=SEED)
psf_gauss = gaussian_mixture_psf(IMG_SIZE, num_components=40, seed=SEED)

fig, axes = plt.subplots(2, 2, figsize=(10, 9))
for ax, psf, title in zip(axes[0], [psf_gauss, psf_phase],
```

```

                                ['Gaussian-mixture PSF\n(lens-like baseline)', 'Random p
im = ax.imshow(psf, cmap='hot')
ax.set_title(title, fontsize=11)
ax.axis('off')
plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

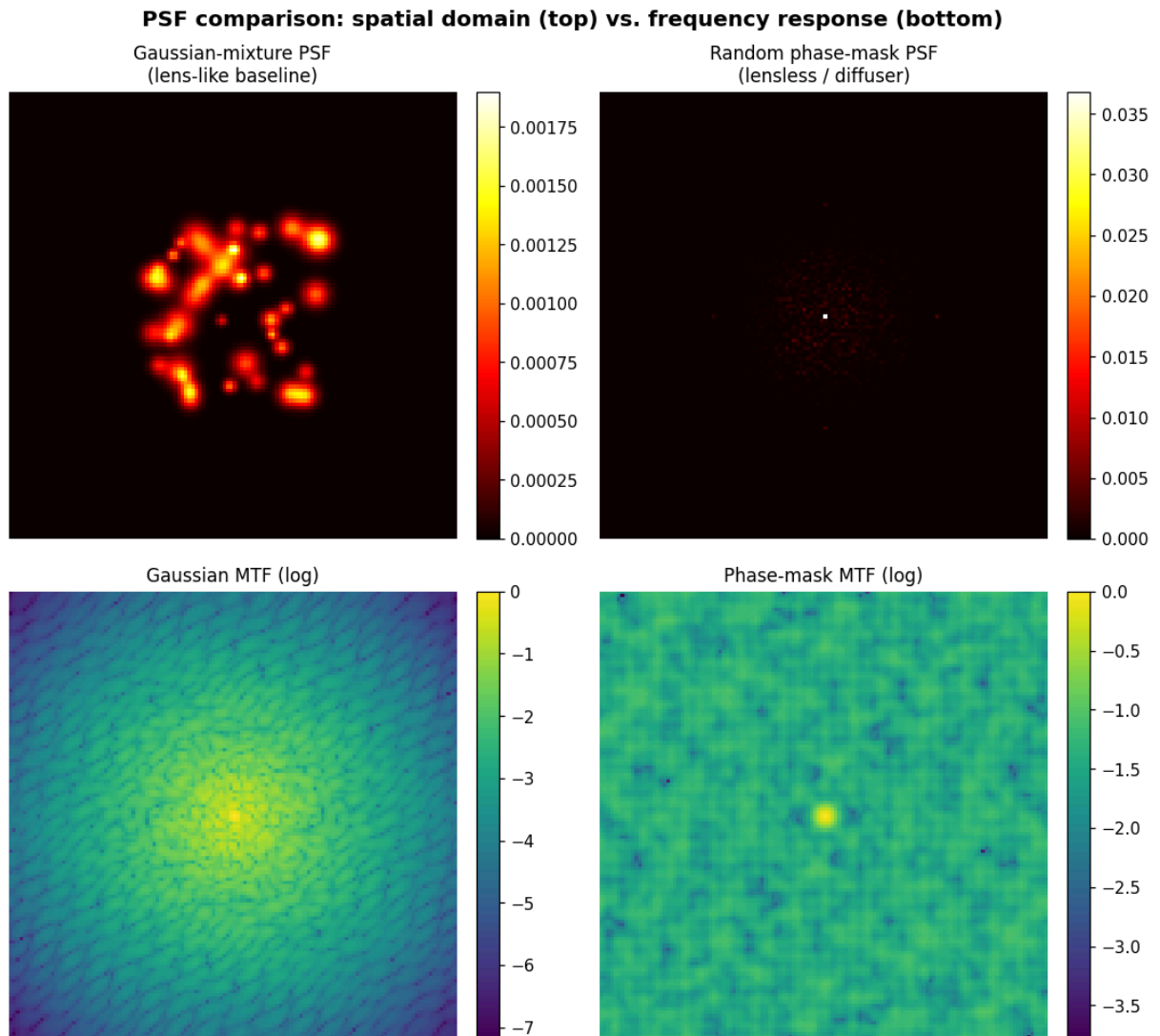
for ax, psf, title in zip(axes[1], [psf_gauss, psf_phase], ['Gaussian MTF (log)', '
psf_n = psf / psf.sum()
mtf = np.abs(np.fft.fftshift(np.fft.fft2(psf_n)))
im = ax.imshow(np.log10(mtf + 1e-8), cmap='viridis')
ax.set_title(title, fontsize=11)
ax.axis('off')
plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

plt.suptitle('PSF comparison: spatial domain (top) vs. frequency response (bottom)'
plt.tight_layout()
plt.show()

print("Notice the phase-mask PSF spreads energy much more broadly in both space and
print("this is exactly what makes lensless imaging possible: information about ever
print("is multiplexed across many sensor pixels, and a broad frequency response mea
print("frequencies are completely lost (no 'spectral nulls').")

# This is the PSF we'll use for the rest of the notebook
psf = psf_phase

```



Notice the phase-mask PSF spreads energy much more broadly in both space and frequency — this is exactly what makes lensless imaging possible: information about every scene point is multiplexed across many sensor pixels, and a broad frequency response means fewer frequencies are completely lost (no 'spectral nulls').

Part 2 — The Forward Imaging Model

Convolution is a per-pixel-costly operation done naively, but is a single element-wise multiplication in the Fourier domain (the convolution theorem):

$$y = \text{PSF} \circledast x + n \quad \Leftrightarrow \quad Y(f) = H(f) \cdot X(f) + N(f)$$

We implement this as a differentiable `torch.nn.Module` so it can be reused both to *simulate* measurements and — since it is differentiable — inside the unrolled network in Part 4.

We also define a few synthetic test scenes (no external image dataset needed) to use throughout the notebook.

```
In [ ]: class ForwardModel(nn.Module):
    '''Lensless imaging forward model:  $y = \text{circular\_conv}(x, \text{psf}) + \text{noise}$ , via FFT.'

    def __init__(self, psf: np.ndarray, noise_std: float = 0.003):
        super().__init__()
        psf_t = torch.from_numpy(np.fft.ifftshift(psf)).float()
        psf_fft = torch.fft.fft2(psf_t)
        self.register_buffer('psf_fft', psf_fft)
        self.register_buffer('psf_fft_conj', torch.conj(psf_fft))
        self.register_buffer('psf_fft_abs_sq', psf_fft.real ** 2 + psf_fft.imag ** 2)
        self.noise_std = noise_std

    def forward(self, x: torch.Tensor, add_noise: bool = True) -> torch.Tensor:
        # x: (B, 1, H, W)
        x_fft = torch.fft.fft2(x.squeeze(1))
        y = torch.fft.ifft2(x_fft * self.psf_fft.unsqueeze(0)).real.unsqueeze(1)
        if add_noise and self.noise_std > 0:
            y = y + self.noise_std * torch.randn_like(y)
        return y

    def make_scene(size=64, kind='shapes', seed=0):
        '''Generate a synthetic test scene. kind in {'checkerboard', 'target', 'shapes'}
        rng = np.random.RandomState(seed)
        img = np.zeros((size, size), dtype=np.float32)

        if kind == 'checkerboard':
            n = 8
            block = size // n
            for i in range(n):
                for j in range(n):
                    if (i + j) % 2 == 0:
                        img[i*block:(i+1)*block, j*block:(j+1)*block] = 1.0

        elif kind == 'target':
            yy, xx = np.mgrid[0:size, 0:size]
            r = np.sqrt((yy - size/2) ** 2 + (xx - size/2) ** 2)
            img = (np.sin(r * 0.9) > 0).astype(np.float32)

        else: # 'shapes' - smooth background + a handful of sharp blobs, natural-image
            base = rng.rand(size, size).astype(np.float32)
            base_t = torch.from_numpy(base)[None, None]
            smooth = F.avg_pool2d(F.pad(base_t, (3, 3, 3, 3), mode='reflect'), 7, stride=1)
            img = smooth[0, 0].numpy()
            img = (img - img.min()) / (img.max() - img.min() + 1e-8)
            for _ in range(4):
                cy, cx = rng.randint(10, size - 10), rng.randint(10, size - 10)
                rad = rng.randint(4, 10)
                yy, xx = np.mgrid[0:size, 0:size]
                mask = (yy - cy) ** 2 + (xx - cx) ** 2 < rad ** 2
                img[mask] = rng.rand()

        return np.clip(img, 0, 1).astype(np.float32)
    from skimage import data, transform, color
```

```

def load_real_scene(name, size):
    img = getattr(data, name)()
    if img.ndim == 3:
        img = color.rgb2gray(img)
    img = transform.resize(img, (size, size), anti_aliasing=True)
    return ((img - img.min()) / (img.max() - img.min() + 1e-8)).astype(np.float32)

# Build the shared forward model used throughout the notebook
forward_model = ForwardModel(psf, noise_std=NOISE_STD).to(device)

sscenes = {name: load_real_scene(name, IMG_SIZE) for name in ['chelsea', 'astronaut', 'coins']}

fig, axes = plt.subplots(2, 3, figsize=(13, 8))
for col, (name, scene) in enumerate(sscenes.items()):
    x = torch.from_numpy(scene[None, None]).to(device)
    y = forward_model(x)

    axes[0, col].imshow(scene, vmin=0, vmax=1)
    axes[0, col].set_title(f'Scene: {name}', fontsize=11)
    axes[0, col].axis('off')

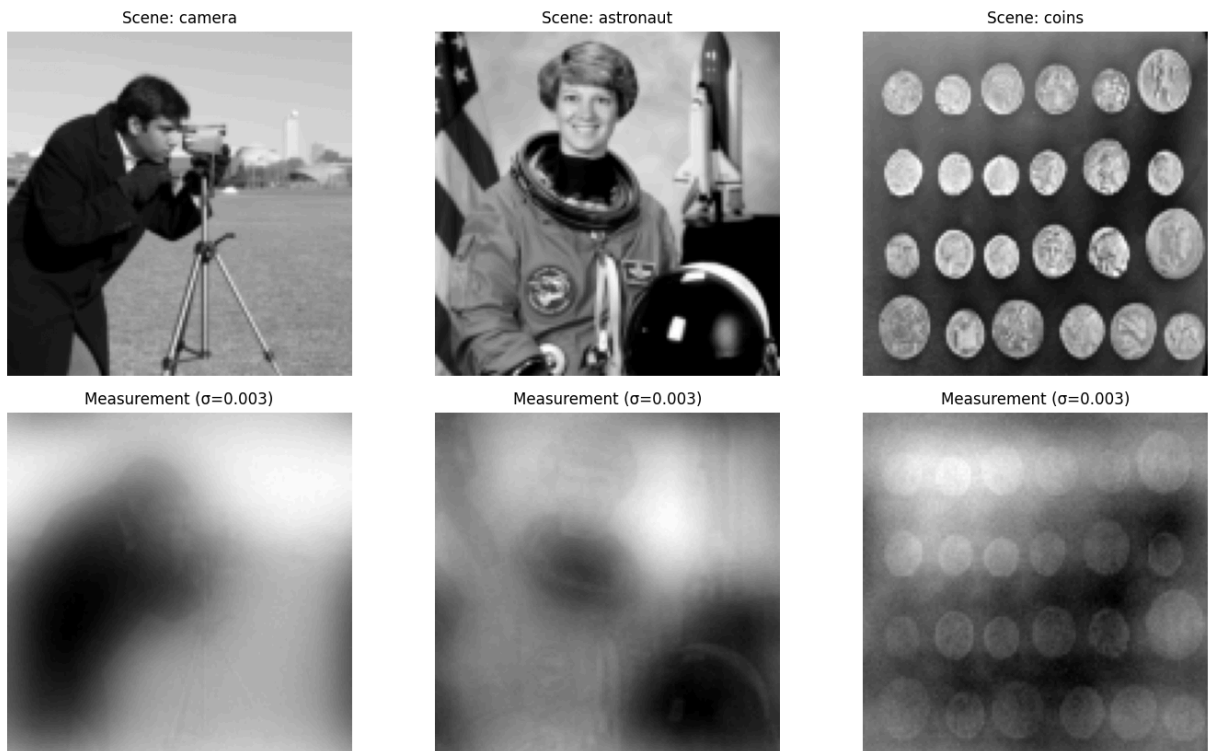
    axes[1, col].imshow(y.squeeze().cpu().numpy())
    axes[1, col].set_title(f'Measurement ( $\sigma$ =NOISE_STD)', fontsize=11)
    axes[1, col].axis('off')

plt.suptitle('Forward model: scene  $\rightarrow$  lensless sensor measurement', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

print("Every measurement looks like an unrecognizable speckle pattern – but all the information is still there, multiplexed across the sensor. Reconstruction is the process of inverting this multiplexing.")

```

Forward model: scene $\rightarrow$ lensless sensor measurement



Every measurement looks like an unrecognizable speckle pattern – but all the scene information is still there, multiplexed across the sensor. Reconstruction is the process of inverting this multiplexing.

Part 3 — Quality Metrics: PSNR & SSIM

Before comparing reconstruction methods we need a way to score them:

- **PSNR** (Peak Signal-to-Noise Ratio, dB) — directly penalizes pixel-wise error; higher is better.
- **SSIM** (Structural Similarity) — compares local luminance, contrast, and structure using a sliding Gaussian window; more aligned with perceived quality.

Both are implemented here directly in PyTorch (no extra dependency).

```
In [ ]: def compute_psnr(x: torch.Tensor, y: torch.Tensor, max_val: float = 1.0) -> float:
    mse = F.mse_loss(x, y).item()
    return 100.0 if mse == 0 else 10 * math.log10(max_val ** 2 / mse)

def _gaussian_kernel(ksize=11, sigma=1.5):
    coords = torch.arange(ksize).float() - ksize // 2
    g = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
    g = g / g.sum()
    return (g[:, None] @ g[None, :])[None, None]

_SSIM_KERNEL = _gaussian_kernel()

def compute_ssim(x: torch.Tensor, y: torch.Tensor, C1=0.01 ** 2, C2=0.03 ** 2) -> float:
    k = _SSIM_KERNEL.to(device=x.device, dtype=x.dtype)
    pad = k.shape[-1] // 2
    mu_x, mu_y = F.conv2d(x, k, padding=pad), F.conv2d(y, k, padding=pad)
    mu_x2, mu_y2, mu_xy = mu_x * mu_x, mu_y * mu_y, mu_x * mu_y
    sigma_x2 = F.conv2d(x * x, k, padding=pad) - mu_x2
    sigma_y2 = F.conv2d(y * y, k, padding=pad) - mu_y2
    sigma_xy = F.conv2d(x * y, k, padding=pad) - mu_xy
    ssim_map = ((2 * mu_xy + C1) * (2 * sigma_xy + C2)) / ((mu_x2 + mu_y2 + C1) * (
    return ssim_map.mean().item()

def grad_xy(u: torch.Tensor):
    dx = torch.zeros_like(u); dx[:, :, :-1, :] = u[:, :, 1:, :] - u[:, :, :-1, :]
    dy = torch.zeros_like(u); dy[:, :, :, :-1] = u[:, :, :, 1:] - u[:, :, :, :-1]
    return dx, dy

def total_variation(x: torch.Tensor) -> torch.Tensor:
    '''Mean anisotropic TV – used both as a training regularizer and to characterize
    dx, dy = grad_xy(x)
    return torch.sqrt(dx ** 2 + dy ** 2 + 1e-8).mean()

# sanity check: identical images
_test = torch.rand(1, 1, 32, 32)
```



```
print(f"PSNR(x, x) = {compute_psnr(_test, _test):.1f} dB (should be very high)")
print(f"SSIM(x, x) = {compute_ssim(_test, _test):.4f} (should be 1.0)")
```

PSNR(x, x) = 100.0 dB (should be very high)

SSIM(x, x) = 1.0000 (should be 1.0)

Part 4 — Classical Reconstruction

4.1 Wiener Deconvolution

The closed-form linear estimator in the frequency domain:

$$\hat{X}(f) = [H^*(f) / (|H(f)|^2 + \lambda)] \cdot Y(f)$$

where H^* is the complex conjugate of the PSF's Fourier transform and λ is a regularization constant that trades off noise amplification against blur. One FFT pair — extremely fast, but with no prior on what natural images look like.

```
In [ ]: def wiener_deconv(y: torch.Tensor, fwd: ForwardModel, reg: float = 1e-3) -> torch.T
        y_fft = torch.fft.fft2(y.squeeze(1))
        X = (fwd.psf_fft_conj / (fwd.psf_fft_abs_sq + reg)) * y_fft
        x_hat = torch.fft.ifft2(X).real.unsqueeze(1)
        return x_hat.clamp(0, 1)

# --- sweep the regularization parameter on the 'shapes' scene ---
scene = scenes['astronaut']
x_gt = torch.from_numpy(scene)[None, None].to(device)
y_meas = forward_model(x_gt)

reg_values = [1e-4, 1e-3, 1e-2, 1e-1]
fig, axes = plt.subplots(1, len(reg_values) + 2, figsize=(20, 3.6))

axes[0].imshow(scene, vmin=0, vmax=1); axes[0].set_title('Ground truth'); axes[0].a
axes[1].imshow(y_meas.squeeze().cpu().numpy()); axes[1].set_title('Measurement'); a

for i, reg in enumerate(reg_values):
    recon = wiener_deconv(y_meas, forward_model, reg=reg)
    p = compute_psnr(recon, x_gt)
    axes[i + 2].imshow(recon.squeeze().cpu().numpy(), vmin=0, vmax=1)
    axes[i + 2].set_title(f'λ={reg}\nPSNR={p:.1f}dB', fontsize=10)
    axes[i + 2].axis('off')

plt.suptitle('Wiener deconvolution: too little regularization amplifies noise, too
plt.tight_layout()
plt.show()
```



4.2 ADMM with a Total-Variation Prior

ADMM lets us add a **prior** knowledge about what natural images look like instead of relying purely on the linear model. We use Total Variation (TV): "*natural images are mostly flat with occasional sharp edges.*"

We solve:

$$\text{minimize}_x \quad (1/2) \|y - \text{PSF} \odot x\|^2 + \lambda \cdot \text{TV}(x)$$

by alternating:

1. **x-update** a data-fidelity least-squares solve, closed-form in the Fourier domain (like Wiener, but pulled toward the current auxiliary variable z).
2. **z-update** apply the TV proximal operator to $x + u$ (this is where the "denoising" happens).
3. **u-update** update the ADMM dual/scaled-residual variable.

The TV proximal operator itself is solved by projected/smoothed gradient descent on

$$\text{prox}_{\text{TV}}(v) = \text{argmin}_x \quad (1/2) \|x - v\|^2 + \lambda \cdot \text{TV}(x)$$

which we verify against autograd below before using it, to make sure the hand-derived gradient is correct.

```
In [ ]: def adjoint_grad(px: torch.Tensor, py: torch.Tensor) -> torch.Tensor:
    '''Adjoint of the forward-difference gradient operator used in grad_xy (i.e. -d
    adjx = -px.clone(); adjx[:, :, 1:, :] += px[:, :, :-1, :]
    adjy = -py.clone(); adjy[:, :, :, 1:] += py[:, :, :, :-1]
    return adjx + adjy

def tv_energy(x, v, lam, eps=1e-6):
    dx, dy = grad_xy(x)
    tv = torch.sqrt(dx ** 2 + dy ** 2 + eps).sum()
    return 0.5 * ((x - v) ** 2).sum() + lam * tv

def tv_prox(v, lam, n_iters=40, eps=1e-4):
    L = 1.0 + lam * 8.0 / (eps ** 0.5) # Lipschitz bound -> safe step size
    step = 1.0 / L
    x = v.clone(); z = v.clone(); t = 1.0
    for _ in range(n_iters):
```

```

    dx, dy = grad_xy(z)
    norm = torch.sqrt(dx ** 2 + dy ** 2 + eps)
    px, py = dx / norm, dy / norm
    grad_e = (z - v) + lam * adjoint_grad(px, py)
    x_new = z - step * grad_e
    t_new = (1 + (1 + 4 * t * t) ** 0.5) / 2
    z = x_new + ((t - 1) / t_new) * (x_new - x)
    x, t = x_new, t_new
    return x

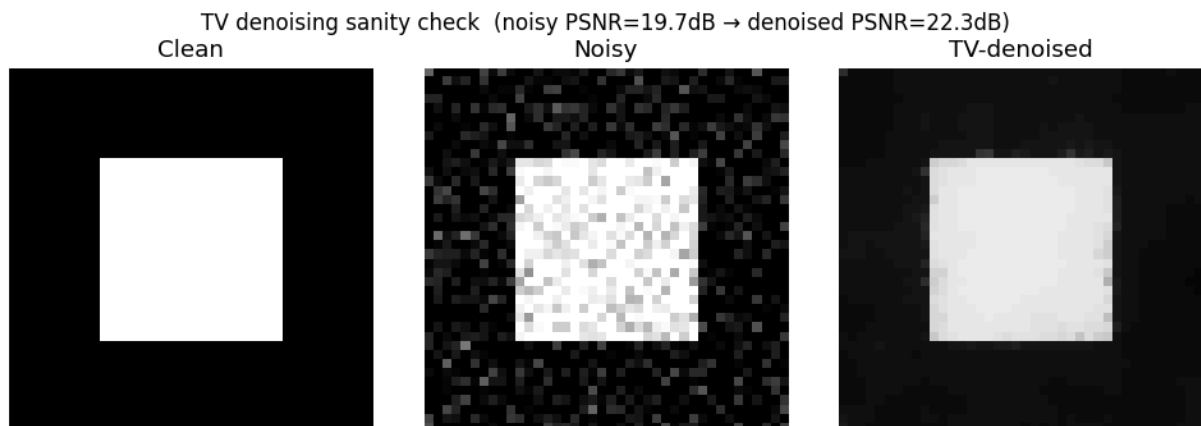
# --- Verify the hand-derived TV gradient against autograd ---
torch.manual_seed(1)
x0 = torch.randn(1, 1, 10, 10, requires_grad=True)
v0 = torch.randn(1, 1, 10, 10)
lam_test = 0.3
E = tv_energy(x0, v0, lam_test)
E.backward()
with torch.no_grad():
    dx, dy = grad_xy(x0)
    norm = torch.sqrt(dx ** 2 + dy ** 2 + 1e-6)
    manual_grad = (x0 - v0) + lam_test * adjoint_grad(dx / norm, dy / norm)
max_diff = (x0.grad - manual_grad).abs().max().item()
print(f"Max difference between autograd and hand-derived TV gradient: {max_diff:.2e}")

# --- Demonstrate TV denoising on a noisy synthetic image ---
clean = torch.zeros(1, 1, 40, 40); clean[:, :, 10:30, 10:30] = 1.0
noisy = (clean + 0.15 * torch.randn_like(clean)).clamp(0, 1)
denoised = tv_prox(noisy, lam=0.15, n_iters=120)

fig, axes = plt.subplots(1, 3, figsize=(9, 3.2))
for ax, img, title in zip(axes, [clean, noisy, denoised], ['Clean', 'Noisy', 'TV-de
    ax.imshow(img.squeeze().numpy(), vmin=0, vmax=1); ax.set_title(title); ax.axis(
plt.suptitle(f'TV denoising sanity check (noisy PSNR={compute_psnr(noisy, clean):.
plt.tight_layout(); plt.show()

```

Max difference between autograd and hand-derived TV gradient: 2.38e-07 (should be ~ 1e-6 or smaller)



```

In [ ]: def admm_reconstruct(y, fwd, rho_start=5.0, rho_end=0.2, lam_tv=0.001, n_iters=30,
    Hc, Habs2 = fwd.psf_fft_conj, fwd.psf_fft_abs_sq
    y_fft = torch.fft.fft2(y.squeeze(1))
    z = wiener_deconv(y, fwd, reg=1e-3)

```

```

u = torch.zeros_like(y)
rhos = np.geomspace(rho_start, rho_end, n_iters)
history = []
for k in range(n_iters):
    rho = float(rhos[k])
    rhs_fft = Hc * y_fft + rho * torch.fft.fft2((z - u).squeeze(1))
    x_new = torch.fft.ifft2(rhs_fft / (Habs2 + rho)).real.unsqueeze(1)
    z_new = tv_prox((x_new + u).clamp(min=0), lam=lam_tv / rho, n_iters=tv_inne
    u = u + x_new - z_new
    z = z_new

    entry = {'iter': k, 'primal_residual': (x_new - z_new).pow(2).mean().item()}
    if ground_truth is not None:
        entry['psnr'] = compute_psnr(z_new, ground_truth)
    history.append(entry)
    if verbose and (k % 5 == 0 or k == n_iters - 1):
        msg = f" ADMM iter {k:3d}: primal_res={entry['primal_residual']:.6f}"
        if 'psnr' in entry:
            msg += f" PSNR={entry['psnr']:.2f} dB"
        print(msg)

return z, history

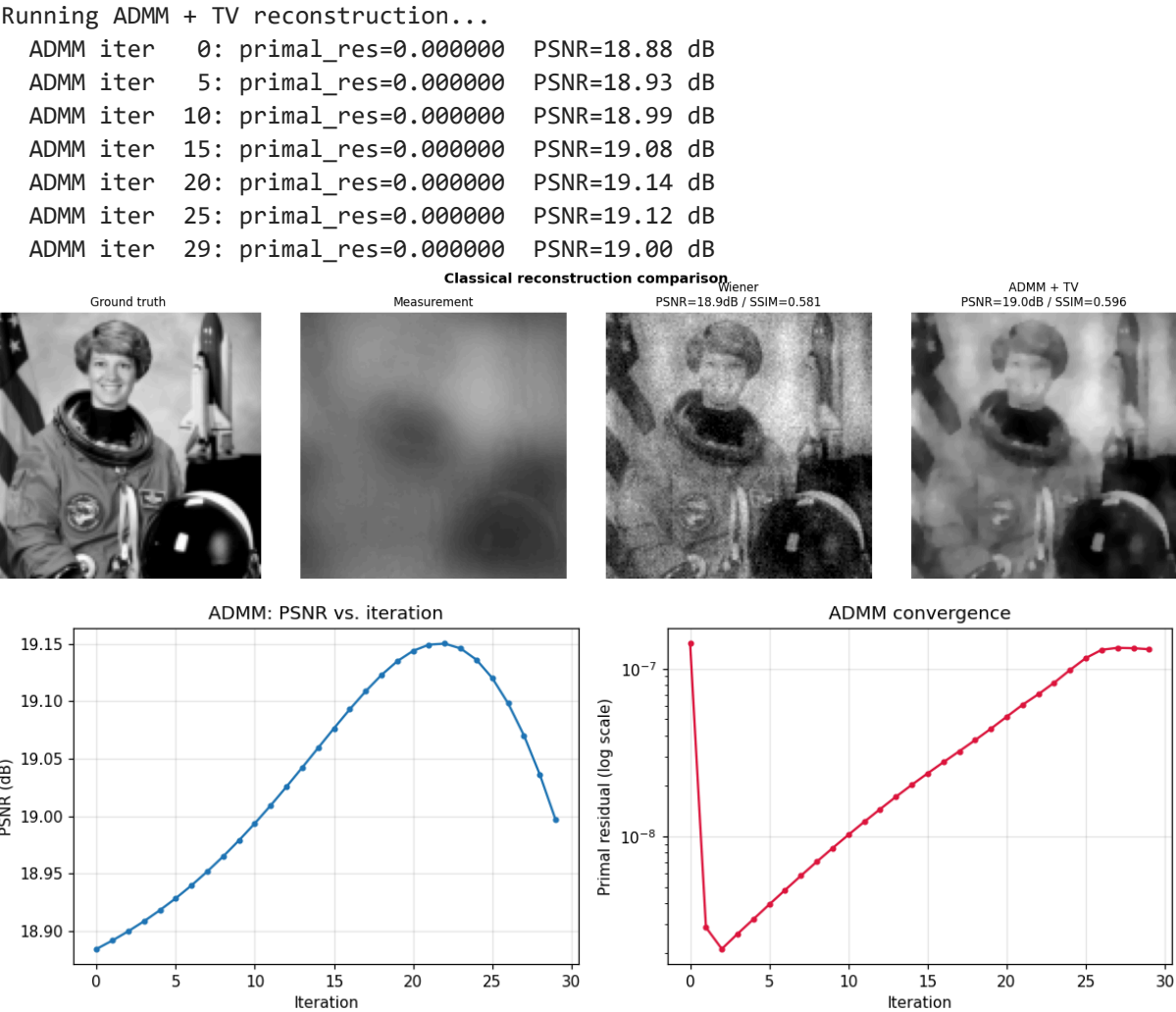
print("Running ADMM + TV reconstruction...")
admm_result, admm_history = admm_reconstruct(y_meas, forward_model, rho_start=5.0,

wiener_result = wiener_deconv(y_meas, forward_model, reg=1e-3)

fig, axes = plt.subplots(1, 4, figsize=(16, 4.2))
for ax, img, title in zip(
    axes,
    [scene, y_meas.squeeze().cpu().numpy(), wiener_result.squeeze().cpu().numpy(),
    ['Ground truth', 'Measurement',
    f'Wiener\nPSNR={compute_psnr(wiener_result, x_gt):.1f}dB / SSIM={compute_ssim(
    f'ADMM + TV\nPSNR={compute_psnr(admm_result, x_gt):.1f}dB / SSIM={compute_ssim
):
    ax.imshow(img, vmin=0, vmax=1 if img.max() <= 1.01 else None)
    ax.set_title(title, fontsize=11); ax.axis('off')
plt.suptitle('Classical reconstruction comparison', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()

# Convergence plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
iters = [h['iter'] for h in admm_history]
ax1.plot(iters, [h['psnr'] for h in admm_history], 'o-', ms=3); ax1.set_xlabel('Ite
ax1.set_title('ADMM: PSNR vs. iteration'); ax1.grid(alpha=0.3)
ax2.semilogy(iters, [h['primal_residual'] for h in admm_history], 'o-', ms=3, color
ax2.set_xlabel('Iteration'); ax2.set_ylabel('Primal residual (log scale)'); ax2.set
plt.tight_layout(); plt.show()

```



Part 5 : Physics-Informed Deep Learning: the Unrolled ADMM Network

Algorithm unrolling turns each iteration of an optimization algorithm into a layer of a neural network. We keep the exact same two-step structure as our classical ADMM solver:

ADMM step	Classical	Unrolled network
x-update	closed-form Fourier solve, fixed ρ	closed-form Fourier solve, learnable ρ per stage
z-update	hand-designed TV proximal operator	learned CNN denoiser (much more expressive than TV)
Depth	30–100+ iterations	4–8 stages (much faster at inference)

Because the x-update is still the *exact* physics-based least-squares solve (using the real PSF), the network can't ignore the optics, it's forced to stay consistent with the forward model at every stage, while the CNN learns whatever prior best matches the training data. This is the

core idea of "physics-informed" reconstruction: model-based structure + learned components, rather than a black-box image-to-image network.

```
In [ ]: class CNNDenoiser(nn.Module):
    '''Small residual CNN that plays the role of the TV proximal / z-update step.'''

    def __init__(self, channels=1, features=24, n_blocks=2):
        super().__init__()
        self.inp = nn.Conv2d(channels, features, 3, padding=1)
        self.blocks = nn.ModuleList([
            nn.Sequential(
                nn.Conv2d(features, features, 3, padding=1), nn.ReLU(inplace=True),
                nn.Conv2d(features, features, 3, padding=1),
            ) for _ in range(n_blocks)
        ])
        self.out = nn.Conv2d(features, channels, 3, padding=1)

    def forward(self, x):
        h = F.relu(self.inp(x))
        for blk in self.blocks:
            h = h + blk(h)
        return x + self.out(h)  # residual prediction of the clean image

class UnrolledStage(nn.Module):
    '''One unrolled ADMM iteration: physics-based x-update + learned CNN z-update.'''

    def __init__(self, init_rho=1.0, features=24, n_blocks=2):
        super().__init__()
        # parameterize rho through softplus so it always stays positive during training
        self.rho_raw = nn.Parameter(torch.tensor(float(np.log(np.exp(init_rho) - 1))))
        self.denoiser = CNNDenoiser(features=features, n_blocks=n_blocks)

    @property
    def rho(self):
        return F.softplus(self.rho_raw) + 1e-3

    def forward(self, y_fft, psf_fft_conj, psf_fft_abs_sq, z, u):
        rho = self.rho
        rhs_fft = psf_fft_conj * y_fft + rho * torch.fft.fft2((z - u).squeeze(1))
        x_new = torch.fft.ifft2(rhs_fft / (psf_fft_abs_sq + rho)).real.unsqueeze(1)
        z_new = self.denoiser(x_new + u)
        u_new = u + x_new - z_new
        return x_new, z_new, u_new

class UnrolledADMM(nn.Module):
    def __init__(self, psf: np.ndarray, num_stages=5, features=24, n_blocks=2, noise_std=0.01):
        super().__init__()
        fwd = ForwardModel(psf, noise_std=noise_std)
        self.register_buffer('psf_fft_conj', fwd.psf_fft_conj)
        self.register_buffer('psf_fft_abs_sq', fwd.psf_fft_abs_sq)
        init_rhos = [1.0 * (1.5 ** i) for i in range(num_stages)]
        self.stages = nn.ModuleList([
            UnrolledStage(init_rho=r, features=features, n_blocks=n_blocks) for r in init_rhos
        ])
        self.noise_std = noise_std
```

```

    ])

    def forward(self, y, return_intermediates=False):
        y_fft = torch.fft.fft2(y.squeeze(1))
        z = y.clone()
        u = torch.zeros_like(y)
        intermediates = []
        for stage in self.stages:
            x_new, z_new, u = stage(y_fft, self.psf_fft_conj, self.psf_fft_abs_sq,
                                   z = z_new)
            if return_intermediates:
                intermediates.append(z.detach().clone())
        return (z, intermediates) if return_intermediates else z

    def print_architecture(self):
        n_params = sum(p.numel() for p in self.parameters())
        print("=" * 55)
        print(f"  Unrolled ADMM Network – {len(self.stages)} stages, {n_params:,} p
        print("=" * 55)
        for i, s in enumerate(self.stages):
            print(f"  Stage {i}: initial rho={s.rho.item():.3f}")
        print("=" * 55)

NUM_STAGES, FEATURES, N_BLOCKS = 5, 24, 2
net = UnrolledADMM(psf, num_stages=NUM_STAGES, features=FEATURES, n_blocks=N_BLOCKS)
net.print_architecture()

```

```

=====
  Unrolled ADMM Network – 5 stages, 106,450 parameters
=====
  Stage 0: initial rho=1.001
  Stage 1: initial rho=1.501
  Stage 2: initial rho=2.251
  Stage 3: initial rho=3.376
  Stage 4: initial rho=5.063
=====

```

Training data

We train on synthetic scenes generated on the fly — random combinations of checkerboards, targets, and blobby "shapes" images — paired with their simulated lensless measurements. This means training data is unlimited and requires no external dataset.

```

In [ ]: from skimage import data as skdata, transform, color

# a pool of varied real photos shipped inside scikit-image itself – no downloads ne
# using a mix of textures/subjects so the network sees more than one kind of conten
_IMAGE_POOL_NAMES = ['camera', 'astronaut', 'chelsea', 'coffee', 'brick', 'grass',
_IMAGE_POOL = []
for _name in _IMAGE_POOL_NAMES:
    _img = getattr(skdata, _name)()
    if _img.ndim == 3:
        _img = color.rgb2gray(_img)
    _img = _img.astype(np.float32)

```



```

_img = (_img - _img.min()) / (_img.max() - _img.min() + 1e-8)
_IMAGE_POOL.append(_img)

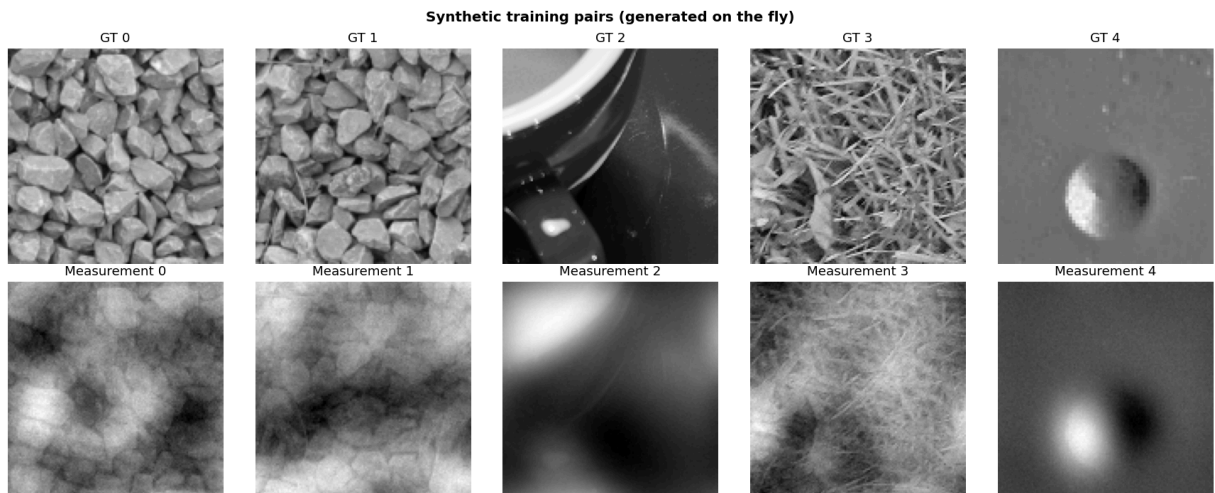
def _random_real_crop(size, rng):
    img = _IMAGE_POOL[rng.randint(len(_IMAGE_POOL))]
    h, w = img.shape
    if h < size or w < size:
        scale = size / min(h, w) * 1.1
        img = transform.resize(img, (int(h * scale) + 1, int(w * scale) + 1), anti_
        h, w = img.shape
    top = rng.randint(0, h - size + 1)
    left = rng.randint(0, w - size + 1)
    crop = img[top:top + size, left:left + size]
    if rng.rand() < 0.5:
        crop = crop[:, ::-1]
    if rng.rand() < 0.5:
        crop = crop[::-1, :]
    return np.ascontiguousarray(crop, dtype=np.float32)

def sample_batch(batch_size, size, seed=None):
    rng = np.random.RandomState(seed)
    imgs = np.stack([_random_real_crop(size, rng) for _ in range(batch_size)])
    return torch.from_numpy(imgs).unsqueeze(1).float()

# preview a few training pairs
preview = sample_batch(5, IMG_SIZE, seed=SEED)
with torch.no_grad():
    preview_y = forward_model(preview.to(device))

fig, axes = plt.subplots(2, 5, figsize=(16, 6.5))
for i in range(5):
    axes[0, i].imshow(preview[i, 0].numpy(), vmin=0, vmax=1); axes[0, i].set_title(
    axes[1, i].imshow(preview_y[i, 0].cpu().numpy()); axes[1, i].set_title(f'Measur
plt.suptitle('Synthetic training pairs (generated on the fly)', fontsize=13, fontwe
plt.tight_layout(); plt.show()

```



```

In [ ]: # ---- Training Loop ----
NUM_EPOCHS      = 100      # bump this up (e.g. 60-100) if you have more time/GPU
STEPS_PER_EPOCH = 25
BATCH_SIZE      = 8
LEARNING_RATE   = 2e-3

```



```

LAMBDA_TV          = 0.01  # weight of the TV regularizer added to the training loss

optimizer = torch.optim.Adam(net.parameters(), lr=LEARNING_RATE)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS)

# fixed validation set so PSNR is comparable across epochs
val_gt = sample_batch(16, IMG_SIZE, seed=999).to(device)
with torch.no_grad():
    val_y = forward_model(val_gt)

train_losses, val_psnrs = [], []
print(f"{'Epoch':>5} | {'Train loss':>11} | {'Val PSNR':>9} | {'Time':>6}")
print("-" * 44)

for epoch in range(1, NUM_EPOCHS + 1):
    net.train()
    t0 = time.time()
    epoch_loss = 0.0

    for step in range(STEPS_PER_EPOCH):
        gt = sample_batch(BATCH_SIZE, IMG_SIZE).to(device)
        with torch.no_grad():
            y = forward_model(gt)

        pred = net(y)
        loss = F.mse_loss(pred, gt) + LAMBDA_TV * total_variation(pred)

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(net.parameters(), max_norm=1.0)
        optimizer.step()
        epoch_loss += loss.item()

    scheduler.step()
    avg_loss = epoch_loss / STEPS_PER_EPOCH
    train_losses.append(avg_loss)

    net.eval()
    with torch.no_grad():
        val_pred = net(val_y)
        val_psnr = compute_psnr(val_pred, val_gt)
    val_psnrs.append(val_psnr)

    if epoch % 5 == 0 or epoch == 1 or epoch == NUM_EPOCHS:
        print(f"{'epoch':>5d} | {'avg_loss':>11.5f} | {'val_psnr':>7.2f} dB | {'time.time()-t0':>6.1f} s")

print("\nTraining complete.")

```

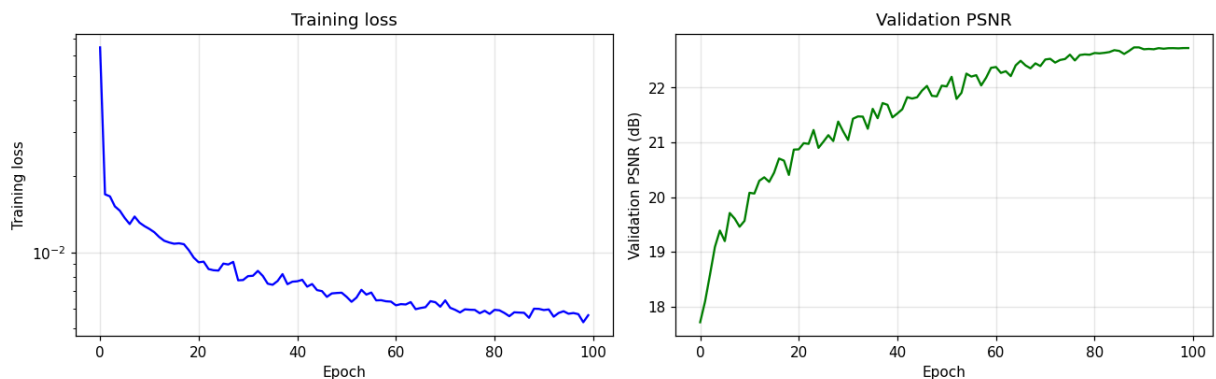
Epoch	Train loss	Val PSNR	Time
1	0.06484	17.72 dB	2.3s
5	0.01461	19.39 dB	1.8s
10	0.01270	19.57 dB	1.7s
15	0.01094	20.28 dB	1.7s
20	0.00952	20.87 dB	1.7s
25	0.00846	20.90 dB	1.7s
30	0.00776	21.20 dB	1.8s
35	0.00751	21.25 dB	1.8s
40	0.00765	21.45 dB	1.8s
45	0.00708	21.82 dB	1.8s
50	0.00691	22.03 dB	1.8s
55	0.00680	22.25 dB	1.8s
60	0.00638	22.36 dB	1.9s
65	0.00595	22.40 dB	2.2s
70	0.00608	22.39 dB	2.0s
75	0.00594	22.52 dB	1.9s
80	0.00570	22.60 dB	1.9s
85	0.00578	22.68 dB	1.9s
90	0.00597	22.73 dB	1.9s
95	0.00584	22.71 dB	1.8s
100	0.00563	22.72 dB	1.9s

Training complete.

```
In [ ]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2))
ax1.plot(train_losses, 'b-'); ax1.set_yscale('log'); ax1.set_xlabel('Epoch'); ax1.s
ax1.set_title('Training loss'); ax1.grid(alpha=0.3)
ax2.plot(val_psnrs, 'g-'); ax2.set_xlabel('Epoch'); ax2.set_ylabel('Validation PSNR
ax2.set_title('Validation PSNR'); ax2.grid(alpha=0.3)
plt.suptitle('Training progress', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()

print("Learned rho (ADMM penalty) per stage after training:")
for i, s in enumerate(net.stages):
    print(f" Stage {i}: rho = {s.rho.item():.4f}")
```

Training progress



Learned rho (ADMM penalty) per stage after training:

Stage 0: rho = 1.5537
 Stage 1: rho = 1.0347
 Stage 2: rho = 1.8235
 Stage 3: rho = 3.0783
 Stage 4: rho = 4.5644

Part 6 — Full Side-by-Side Comparison

Now we run all three methods — Wiener, ADMM+TV, and the trained Unrolled ADMM network — on a fresh test scene and compare quality and runtime.

```
In [ ]: net.eval()
test_scene = load_real_scene("astronaut", 128)
x_test = torch.from_numpy(test_scene)[None, None].to(device)
with torch.no_grad():
    y_test = forward_model(x_test)

results = {}

t0 = time.time()
r_wiener = wiener_deconv(y_test, forward_model, reg=1e-3)
results['Wiener'] = (r_wiener, time.time() - t0)

t0 = time.time()
r_admm, _ = admm_reconstruct(y_test, forward_model, rho_start=5.0, rho_end=0.2, lam
results['ADMM + TV'] = (r_admm, time.time() - t0)

t0 = time.time()
with torch.no_grad():
    r_learned = net(y_test)
results['Unrolled ADMM (learned)'] = (r_learned, time.time() - t0)

print(f"{'Method':<26} | {'PSNR (dB)':>10} | {'SSIM':>7} | {'Time (s)':>9}")
print('-' * 62)
for name, (recon, dt) in results.items():
    p, s = compute_psnr(recon, x_test), compute_ssim(recon, x_test)
    print(f"{'name':<26} | {p:9.2f} | {s:6.4f} | {dt:8.4f}")

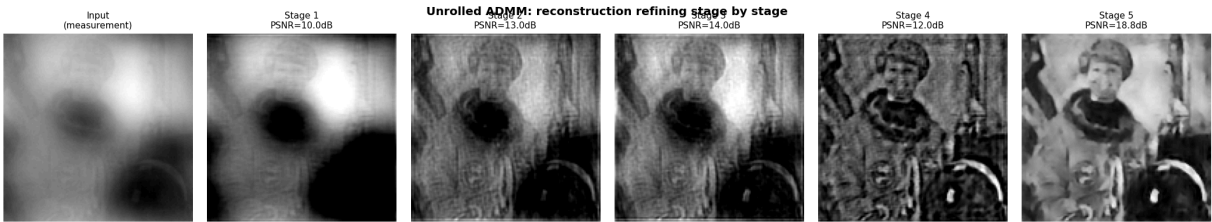
fig, axes = plt.subplots(1, 2 + len(results), figsize=(4 * (2 + len(results)), 4.2))
axes[0].imshow(test_scene, vmin=0, vmax=1); axes[0].set_title('Ground truth'); axes
axes[1].imshow(y_test.squeeze().cpu().numpy()); axes[1].set_title('Measurement'); a
for ax, (name, (recon, dt)) in zip(axes[2:], results.items()):
    p, s = compute_psnr(recon, x_test), compute_ssim(recon, x_test)
    ax.imshow(recon.squeeze().cpu().numpy(), vmin=0, vmax=1)
    ax.set_title(f'{name}\nPSNR={p:.1f}dB SSIM={s:.3f}', fontsize=10)
    ax.axis('off')
plt.suptitle('Reconstruction method comparison', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```

Method	PSNR (dB)	SSIM	Time (s)
Wiener	18.89	0.5852	0.0007
ADMM + TV	18.50	0.5257	3.4204
Unrolled ADMM (learned)	18.85	0.6246	0.0126



```
In [ ]: # Visualize how the unrolled network's estimate evolves stage by stage
net.eval()
with torch.no_grad():
    _, stage_outputs = net(y_test, return_intermediates=True)

fig, axes = plt.subplots(1, len(stage_outputs) + 1, figsize=(3.2 * (len(stage_outputs) + 1), 3.2 * (len(stage_outputs) + 1)))
axes[0].imshow(y_test.squeeze().cpu().numpy()); axes[0].set_title('Input\n(measurement)')
for i, out in enumerate(stage_outputs):
    p = compute_psnr(out, x_test)
    axes[i + 1].imshow(out.squeeze().cpu().numpy(), vmin=0, vmax=1)
    axes[i + 1].set_title(f'Stage {i+1}\nPSNR={p:.1f}dB', fontsize=10)
    axes[i + 1].axis('off')
plt.suptitle('Unrolled ADMM: reconstruction refining stage by stage', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```



Summary

Method	Idea	Pros	Cons
Wiener	Closed-form linear inverse filter in Fourier domain	Extremely fast, single FFT pair	No image prior; noise/blur trade-off is crude
ADMM + TV	Iterative optimization with a hand-designed sparsity prior	Principled, works without any training data, often the best classical baseline	Slow (needs many iterations), TV assumption is simplistic (over-smooths textures)
Unrolled ADMM (learned)	Same iterative structure as ADMM, but the prior (z-update) is a trained CNN and hyperparameters are learned	Combines physics (exact forward model) with a much richer learned prior; fast at inference (few stages)	Needs training data/compute; quality depends on training scene diversity

Ideas for extending this notebook

- Increase `IMG_SIZE` , `NUM_STAGES` , `FEATURES` , and training epochs for higher-fidelity results.
- Swap in a real captured PSF (e.g. from a calibration image of a point source) instead of the simulated one.
- Extend to 3-channel (RGB) scenes and measurements.
- Add a perceptual (VGG-feature) loss term alongside MSE + TV.
- Try training with variable noise levels for a more noise-robust network.
- Compare against a plain black-box U-Net (no physics in the loop) to see how much the physics-informed structure actually helps.

In []: